COMBINED PLANNING REPORT

naijaDNA Scaling and Streaming Budget Report

This combined report brings together the corrected scaling budget guidance and the streaming platform budget recommendation. It keeps daily active users written out, corrects the earlier caching ambiguity, and adds a platform recommendation for live streaming.

Date: 2026-04-07 Audience: anonymous readers and signed-in readers Traffic focus: reads, polls, live updates, streams

CURRENT APP COMPUTE

0.5 CPU / 512 MB

Single instance, no autoscaling

CURRENT DATABASE

Render Postgres Free

Not suitable for sustained public production

STREAMING STACK FIT

LiveKit already integrated

Lowest rewrite risk for live sessions

All user-capacity figures below are directional estimates for this codebase, not vendor guarantees. They assume Redis caching for read-heavy endpoints and static assets served through Cloudflare.

Corrected Cost Logic

1. A cached setup is usually the same price or slightly more expensive as a monthly bill.
2. A cached setup usually serves more users before the next upgrade.
3. That means it is cheaper per 1,000 daily active users, even if the bill itself is not lower.

1. Render Shape: Near-Term Production

Cloudflare Pages, Render Web, Render Postgres, Upstash Redis, optional worker later.

Deployment Shapes

2. AWS Lightsail Shape: Medium-Term Growth

Cloudflare Pages, 2-3 app instances, load balancer, managed PostgreSQL, Upstash Redis, Cloudflare R2.

3. AWS ECS / RDS Shape: Long-Term Scale

Cloudflare Pages, ECS / Fargate API, worker service, RDS PostgreSQL, Redis, Application Load Balancer.

Budget Ladder With Recommended User Counts

Stage	Deployment shape	Monthly bill	Daily active users	Concurrent readers	Cost / 1,000 users
Stage 1	Render lean, minimal cache	about $14-$20	up to 2,000	30-50	about $7-$10
Stage 2	Render lean, with caching discipline	about $14-$24	up to 3,000	50-75	about $4.67-$8
Stage 3	Render safer	about $54-$100	up to 8,000	75-200	about $6.75-$12.50
Stage 4	AWS Lightsail base	about $67-$72	up to 15,000	200-350	about $4.47-$4.80
Stage 5	AWS Lightsail growth	about $116-$121	up to 40,000	350-800	about $2.90-$3.03
Stage 6	AWS Lightsail stronger spike tier	about $170-$188	up to 80,000	800-1,500	about $2.13-$2.35
Stage 7+	AWS ECS / RDS serious production	about $150-$340 lean; $350-$800+ mature	100,000+	1,500-5,000+	about $1.50-$3.40 at 100,000+

Recommended User Targets Per Shape

Shape	Recommended user target	Why this is the planning target
Render lean	up to 2,000 daily active users	Cheapest entry point, but single-instance limits stay tight.
Render lean with caching discipline	up to 3,000 daily active users	Better cache hit rate postpones the compute and database upgrade.
Render safer	up to 8,000 daily active users	Good near-term production posture before you need multiple app instances.
AWS Lightsail base	up to 15,000 daily active users	First strong multi-instance architecture for this codebase.
AWS Lightsail growth	up to 40,000 daily active users	Best cost-to-capacity point for a growing news product.
AWS Lightsail stronger spike tier	up to 80,000 daily active users	Better suited to live news spikes and denser poll traffic.
AWS ECS / RDS	100,000+ daily active users	Best when scale and reliability needs outweigh simplicity.

Streaming Budget by Platform

Platform	Best fit	Entry budget	Growth budget	Recommendation
LiveKit Cloud	Interactive live sessions, host plus guests, viewer participation	about $0 testing or about $50/month to start shipping	about $500/month Scale plan, plus usage above quotas	Best immediate choice for this codebase
Mux	Large one-to-many broadcasts with mostly passive viewers	about $0 to start, with 100,000 monthly delivery minutes free	usage-based; 200,000 delivered minutes at 720p is about $80 after the free tier, plus live input at $0.025/minute	Best if you pivot to broadcast-first viewing
Daily	Interactive video and audio calls with simple participant-minute pricing	about $0 for the first 10,000 minutes each month	about $0.004 per participant-minute at lower tiers, plus optional streaming or recording charges	Good product, but not the best fit when LiveKit is already integrated
Agora	Interactive streaming with larger enterprise networking needs	free for the first 10,000 minutes in some usage bands	pricing gets complex quickly; proxy-heavy interactive streaming can hit a $500 monthly minimum base fee	Not the first recommendation for this codebase

Simple Streaming Budget Scenarios

Scenario	Rough monthly usage	Most suitable platform	Budget guidance
Small editorial live sessions	1-4 live sessions per month, tens of viewers, interactive host presence	LiveKit	Keep the existing LiveKit integration. Expect the Build or Ship tier to be the right place to start.
Regular community streams	Weekly interactive shows with 50-200 concurrent viewers	LiveKit	Budget around LiveKit Ship or Scale depending on viewer concurrency and bandwidth profile.
Large passive breaking-news broadcasts	Hundreds or thousands of mostly passive viewers	Mux or a LiveKit plus Mux hybrid	Viewer delivery becomes the main cost, and a broadcast-first platform usually wins on cost efficiency.

Platform Recommendation

Recommended now: Keep LiveKit

For the current codebase, LiveKit is the right platform to keep using for streaming because it is already integrated in both backend and Flutter; the current stream feature is interactive, not just passive broadcast; changing providers now would create avoidable backend and client rewrite work; and LiveKit already fits the current room-token architecture and viewer join flow.

Recommended later: add Mux if streams become broadcast-heavy

If the product evolves into large one-to-many broadcast events, the best medium-term architecture is likely LiveKit for host, backstage, or guest coordination and Mux for large-scale viewer delivery, while keeping the rest of the application flow unchanged.

Recommendation in one line: keep LiveKit now, because it matches the existing implementation; only move to Mux or a LiveKit plus Mux hybrid when the audience becomes mostly passive and stream viewer counts materially outgrow the economics of interactive WebRTC delivery.

Architecture Notes For This Codebase

1. Keep frontend delivery on Cloudflare Pages.
2. Move scheduled ingestion and the scheduler out of the API process before adding multiple API instances.
3. Keep Redis as a shared response cache for homepage reads, poll reads, category reads, live-update reads, and stream-related read endpoints where appropriate.
4. Upgrade off free Postgres immediately if the app is meant to serve real users.
5. Move media to Cloudflare R2 so article, live-update, and stream-related images do not rely on app-disk storage.

Streaming Rollout Path

1. Continue with LiveKit for the current interactive streaming feature.
2. Measure actual monthly stream minutes, concurrency, and average viewer counts.
3. If streams stay interactive, scale within LiveKit.
4. If broadcasts become large and mostly passive, introduce Mux for viewer delivery while keeping LiveKit for host-side interactivity.

Sources

1. Render pricing: https://render.com/pricing/
2. Render Key Value documentation: https://render.com/docs/key-value
3. Upstash Redis pricing: https://upstash.com/pricing
4. Upstash Redis limits and pricing detail: https://upstash.com/docs/redis/overall/pricing
5. Cloudflare Workers pricing: https://developers.cloudflare.com/workers/platform/pricing/
6. Cloudflare Pages Functions pricing: https://developers.cloudflare.com/pages/functions/pricing/
7. Amazon Lightsail pricing: https://aws.amazon.com/lightsail/pricing/
8. AWS Fargate pricing: https://aws.amazon.com/fargate/pricing/
9. AWS RDS PostgreSQL pricing: https://aws.amazon.com/rds/postgresql/pricing/
10. LiveKit pricing calculator and plans: https://livekit.io/pricing/calculator
11. LiveKit pricing guide for video conference and livestream: https://livekit.io/field-guides/guide/estimating-pricing-video-conference-livestream
12. LiveKit scale and WebRTC connection limits: https://livekit.io/field-guides/guide/plan-livekit-scale
13. Mux pricing: https://www.mux.com/pricing
14. Mux live input and delivery pricing details: https://www.mux.com/live and https://www.mux.com/docs/pricing/video
15. Daily pricing: https://www.daily.co/pricing/video-sdk/
16. Agora interactive live streaming pricing: https://www.agora.io/en/pricing/interactive-live-streaming/

Practical Summary

If you want the lowest-risk next step, keep LiveKit and budget streaming as an extension of the current stack. If your future newsroom strategy shifts to large passive live events, treat Mux as the broadcast delivery option to evaluate next. For the application stack overall, the strong cost-to-capacity middle ground remains AWS Lightsail growth once you outgrow a safer Render setup.